

LOST IN THE MIDDLE - Findings Report

Sagnik Chandra

August 3, 2026

Contents

1	Factoid LIM Findings	2
1.1	Overview	2
1.2	Methodology	2
1.3	Multi-Document Question Answering	2
1.4	Synthetic Key-Value Retrieval	3
1.5	Findings	4
2	Influence of Attention Sinks	8
2.1	Overview	8
2.2	Impact of Attention Sinks in Primacy Phenomenon	8
2.3	Methodology	8
2.4	Findings	9
3	Non-Factoid LIM Findings	12
3.1	Overview	12
3.2	Methodology	12
3.3	Findings	16

1 Factoid LIM Findings

1.1 Overview

This section covers findings related to factoid queries, which are identified by short, fact-based questions with answers defined across multiple documents.

1.2 Methodology

We reimplemented the methodology defined in Liu et al.[3], where 2 types of experiments have been mentioned:

1. Multi-Document Question Answering
2. Synthetic Key-Value Retrieval

1.3 Multi-Document Question Answering

In Multi-Document Question Answering, models are required to find relevant information within an input context and use it to answer a particular question. To check the performance of the models, we make controlled changes to the length of the input context and the position of the relevant information and measure changes in task performance [3].

Parameter	Description
Source Dataset	NaturalQuestions-Open (Questions from Google Search queries, and human-annotated answers from Wikipedia), available in the GitHub repository of Liu et al. [3]
Discriminator Document Generation	Contriever, fine-tuned on MS-MARCO, to retrieve the $k - 1$ Wikipedia chunks that are most relevant to the query and do not contain any of the NaturalQuestions-annotated answers [3]
Dataset Size	2655
Individual Input Context Lengths (Document count) Used	10 ($\sim$ 2K tokens), 20 ($\sim$ 4K tokens), 30 ($\sim$ 6K tokens)
Model Used	Llama 3.1 8B Instruct (Context Length: 128K)

Table 1: Experimental setup details.

Evaluation

For each model output, we determine whether the LLM has correctly retrieved the answer corresponding to a given query by comparing it against the ground-truth answer(s) provided in the dataset. Before comparison, both the generated response and the ground-truth answer are normalized following the SQuAD evaluation protocol. Specifically, all text is converted to lowercase, punctuation is removed, the articles *a*, *an*, and *the* are discarded, and extra whitespace is collapsed. The normalized ground-truth answer is then checked as a contiguous substring within the normalized model response. If any of the normalized ground-truth

answers is present in the normalized prediction, the output is considered correct; otherwise, it is considered incorrect.:

$$n_i = \begin{cases} 1 & \text{ground truth answer} \in \text{normalized prediction string} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The ‘‘Accuracy’’ of the model is then calculated as:

$$A = \frac{\sum_i n_i}{N} * 100 \quad (2)$$

N being the total number of test cases.

1.4 Synthetic Key-Value Retrieval

As suggested by Liu et al.[3], the Synthetic Key Value Retrieval task, serves as a minimal testbed to check for the basic ability of any LLM model to retrieve information in a particular given context. This is therefore a much miniaturised version of the Multi-Document QA task.

In Synthetic Key Value Retrieval Task, we give a set of string serialized JSON objects to an LLM model as context. These JSON objects contain randomly generated, and unique, key-value pairs, where each if the keys and values are unique, randomly generated UUIDs.

We query the model with a key UUID which exists within the aforementioned JSON object, and expect the LLM to provide us with the appropriately mentioned value UUID.

To check for the Lost in the Middle effect, we choose a set of position indexes within the JSON object, and we query the key UUID located in the key-value pair, at a particular index to the model. The chosen indexes are uniformly moved across the entire length of the main JSON object, which allows us to capture the retrieval capacity of a model across the entire length of a JSON object.

Parameter	Description
Source Dataset	KV JSON dataset available on the GitHub repository for Liu [3], available in the GitHub repository of Liu et al. [3]
Dataset Size	500
Individual Input Context Lengths (KV Pairs) Used	75 (~ 4K tokens), 140 (~ 8K tokens), 300 (~ 16K tokens)
Model Used	Llama 3.1 8B Instruct (Context Length: 128K)

Table 2: Experimental setup details.

Evaluation

For any particular model output, we check whether the LLM has been able to accurately retrieve the correct value UUID for the queried key UUID. we refer to the key-value pair of the queried key (also called gold key-value pair), to get the ground truth. For evaluation, we use the following metric:

$$n_i = \begin{cases} 1 & \text{if generated UUID} = \text{gold_kv['value']} \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

The “Accuracy” of the model is then calculated as:

$$A = \frac{\sum_i n_i}{N} * 100 \quad (4)$$

N being the total number of test cases.

1.5 Findings

Synthetic KV Retrieval Task

The results of the Synthetic Key Value Retrieval task, do not correlate with the findings of Liu et al. [3]. The characteristic U-shaped curve that is the primary indicator for the Lost in the Middle effect is not visible in the curve for all cases tested.

The Multi-Document QA results showcase a very weak U-shaped curve, which becomes more apparent, as the document count in the context increase.

KV Pair Count	Gold KV Position	Accuracy	Accuracy (%)
75	1	492/500	98.40
	25	493/500	98.60
	50	494/500	98.80
	75	454/500	90.80
140	1	498/500	99.60
	35	499/500	99.80
	70	495/500	99.00
	105	494/500	98.80
	140	470/500	94.00
300	1	498/500	99.60
	50	500/500	100.00
	100	498/500	99.60
	150	497/500	99.40
	200	498/500	99.60
	250	494/500	98.80
	300	475/500	95.00

Table 3: Synthetic KV retrieval accuracy by gold KV-pair position, across three total KV-pair counts.

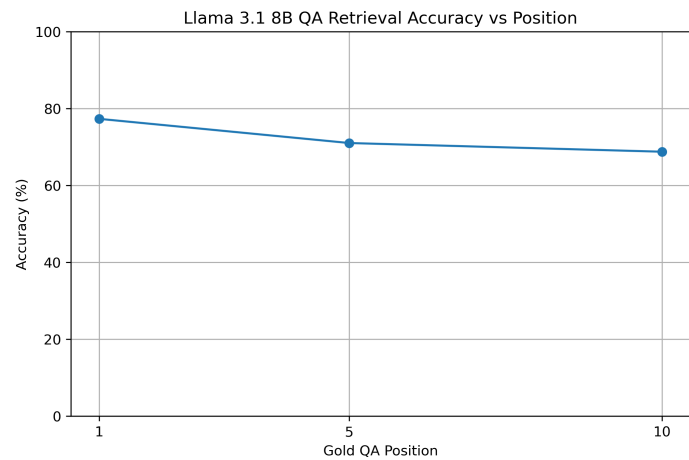
Document Count	Gold Document Position	Accuracy	Accuracy (%)
10	1	2053/2655	77.32
	5	1886/2655	71.03
	10	1826/2655	68.78
20	1	1961/2655	73.86
	5	1771/2655	66.70
	10	1758/2655	66.21
	15	1730/2655	65.16
	20	1775/2655	66.85
30	1	1926/2655	72.54
	5	1722/2655	64.86
	10	1737/2655	65.42
	15	1723/2655	64.89
	20	1712/2655	64.48
	25	1720/2655	64.78
	30	1786/2655	67.27

Table 4: Multi-Document QA accuracy by gold document position, across three total document counts.

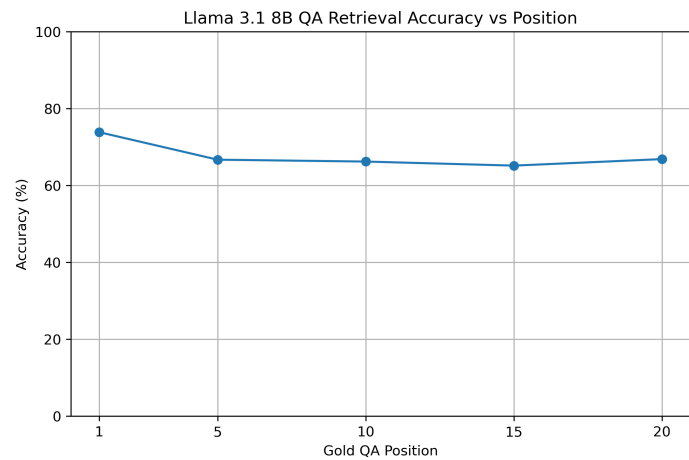
Both the results are consistent with the findings of Veseli et al. [7], who showcased that the Lost in the Middle effect weakens as the ratio of the context window of the model and the token length of the input prompt (defined by L_{rel}) is less than 0.5 (Basically when the input prompt is occupying less than half of the context window of the model). Here, for the 3 cases, the values of L_{rel} is 32, 16 and 8 respectively, which much higher than the mentioned 0.5 [7].

A further analysis with models having smaller context windows is yet to be done.

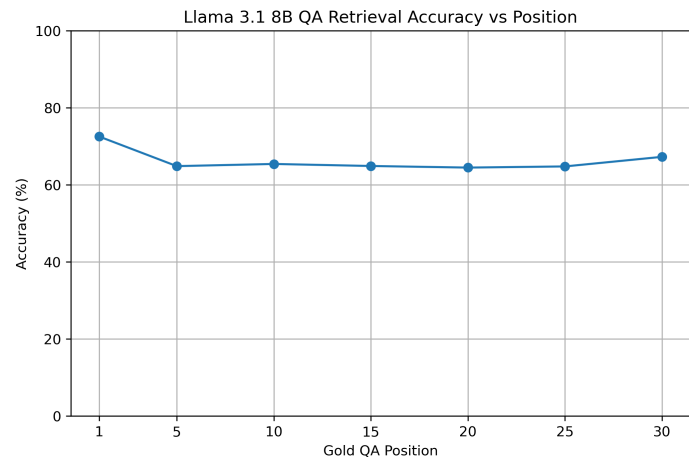
Multi-Document Question Answering Results



(a) 10 Documents



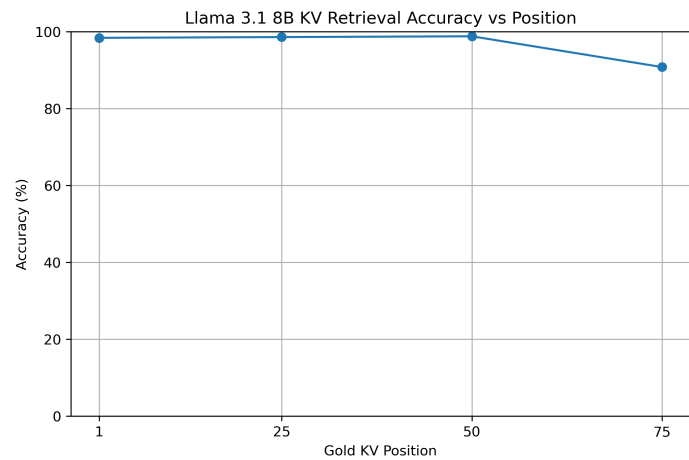
(b) 20 Documents



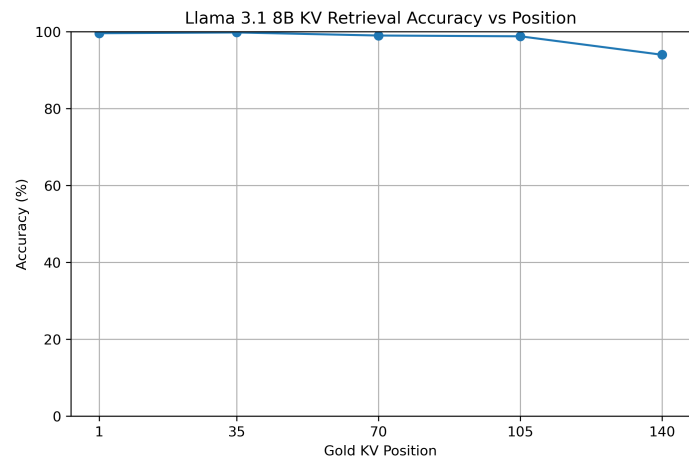
(c) 30 Documents

Figure 1: Multi-Document Question Answering Results for Mistral-7B-Instruct-v0.3.

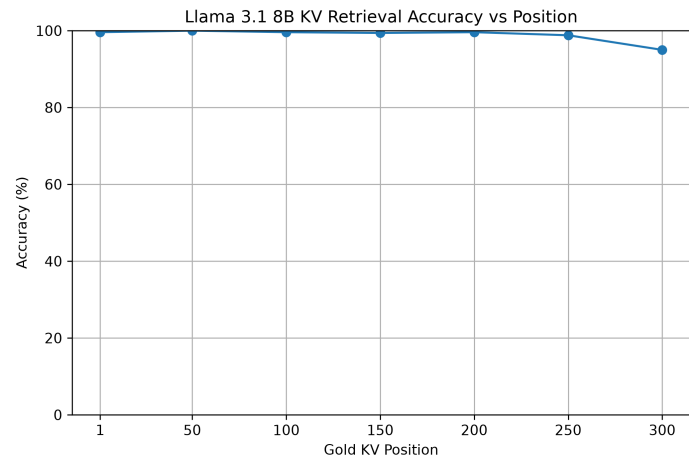
Synthetic KV Retrieval Task Results



(a) 75 KV Pairs



(b) 140 KV Pairs



(c) 300 KV Pairs

Figure 2: Synthetic KV retrieval task results for Mistral-7B-Instruct-v0.3.

2 Influence of Attention Sinks

2.1 Overview

This sections tries to analyse the existence of Attention Sinks in the Multi-Document QA setup, that was defined in the previous section. The main reasoning behind this task is to verify the claims of Salvatore et al. [5], where identification and thus usage of dropouts in layers which showcased attention sinks apparently reduced the Primacy effect, which indicates a dependency between the Primacy effect, and the existence of Attention Sinks at particular layers of a model.

2.2 Impact of Attention Sinks in Primacy Phenomenon

The Lost-in-the-Middle Effect, as a whole, is identified by 3 main indicators, a high accuracy of model at the start and the end, defined as Primacy, and Recency phenomenon respectively.[3], and a charecteristic U-shaped curve in the middle [3]. The reasoning behind the existence of the Recency phenomenon, is apparently shaped by the short-term memory demand of an LLM [5], while the Primacy phenomien arises from a combination of uniform long-term retrieval demand and the causal masking phenomenon of LLMs [5], which ultimately creates attention sinks.

2.3 Methodology

We use the Multi-Document QA setup that we defined in the previous section. We have specifically worked with the 10 document setup, where the gold document count is 1 and non-gold document count is 9.

For determining the existence of attention sinks, we use the methodology defined by Gu et al. [2] [5], which proposed a threshold-based method for identifying and measuring attention sinks across transformer layers and heads. For each attention head h in layer l , the importance score for the k -th token is defined as the average attention it receives across all tokens from position k to the end of the sequence of length T :

$$\alpha_h^l(k) = \frac{1}{T - k + 1} \sum_{i=k}^T A_{i,k}^l$$

We aggregate this importance score across all heads H and all layers L of the model to obtain a single sink score for each token position k :

$$\text{Sink}(k) = \frac{1}{L} \sum_{l=1}^L \frac{1}{H} \sum_{h=1}^H \alpha_h^l(k)$$

A token position k is identified as an attention sink if $\text{Sink}(k)$ exceeds a chosen threshold ϵ , which we determine by inspecting the resulting plot of $\text{Sink}(k)$ across token positions.

We vary the position of the gold document across the context to analyse whether this affects the attention sinks formed in any way.

For this setup, we chose to analyse the existence of attention sinks on the first generated token from a model. We are yet to see the behaviour when for subsequent generated tokens. We have worked with 2 different cases:

Parameter	Description
ϵ value chosen	0.3
Total prompt count tested on	1, 2
Model Used	Llama 3.1 8B Instruct (Context Length: 128K)

Table 5: Experimental setup details.

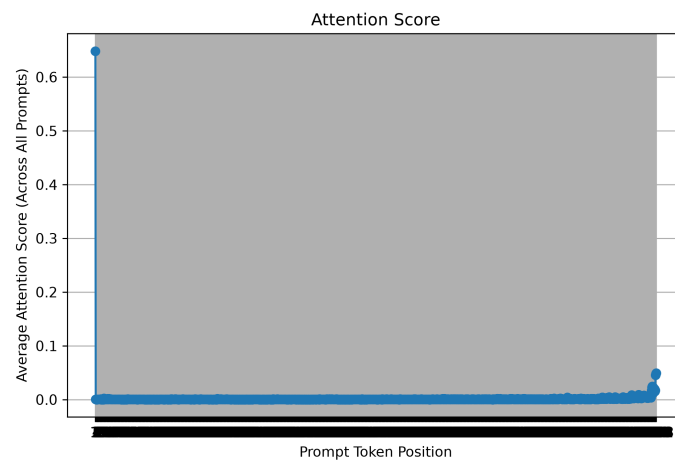
2.4 Findings

Our analysis indicates a very strong attention sink across the cases observed, as for every scenario, we saw that $\text{Sink}(k)$ for the first token to have crossed the threshold ϵ while the subsequent tokens have much smaller $\text{Sink}(k)$ values and have not crossed the threshold.

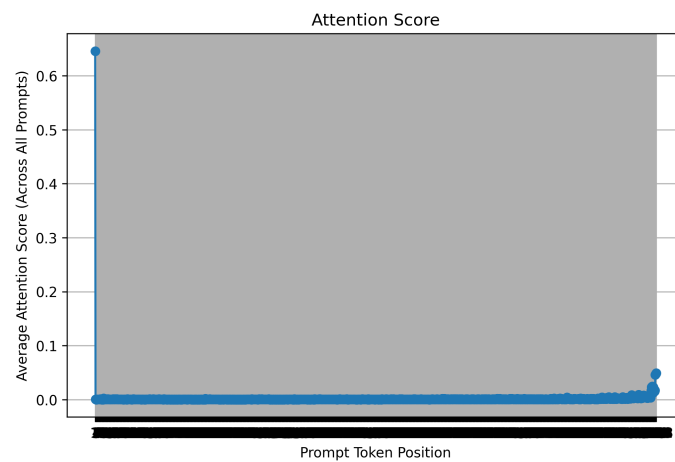
This confirms that the Multi-Document QA setup does showcase attention sinks, which as per the findings of Salvatore et al. [5], implies that Multi-Document QA setup that was defined in the previous section should showcase primacy phenomena. As mentioned at the end of the previous section, working with other types of models might allow us to see this behaviour, a task that will be done after going through the pending task mentioned in the previous section.

Seeing the behaviour of the model and the correlation with the Primacy condition when introducing dropouts on layers which showcase attention sinks, to check with the findings of Salvatore et al. [5] is pending, and will be attended to once the previously mentioned pending tasks are completed.

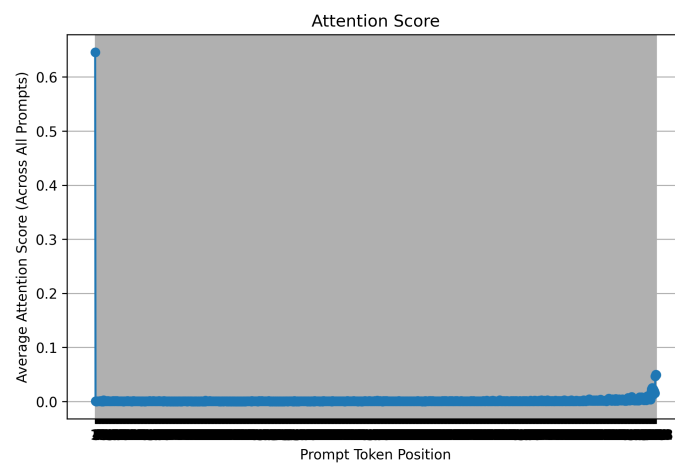
Attention Sinks Analysis (1 prompt used)



(a) Gold document at 1st position



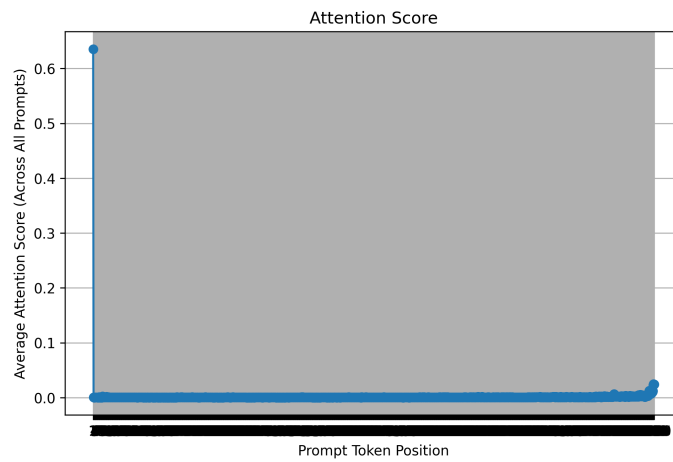
(b) Gold document at 5th position



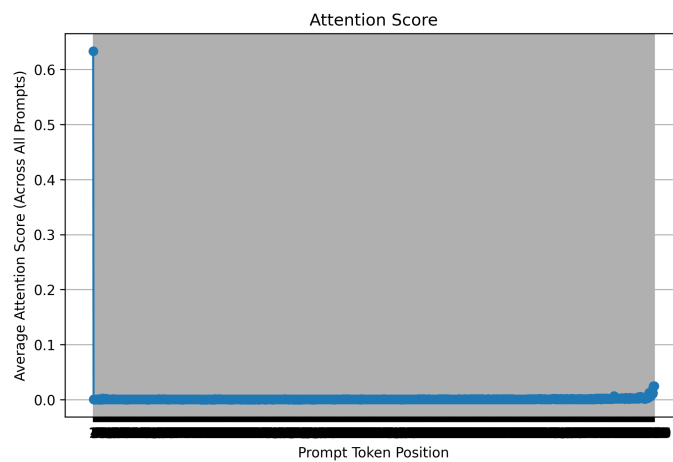
(c) Gold document at 10th position

Figure 3: Attention Sink analysis results, here only 1 prompt has been used

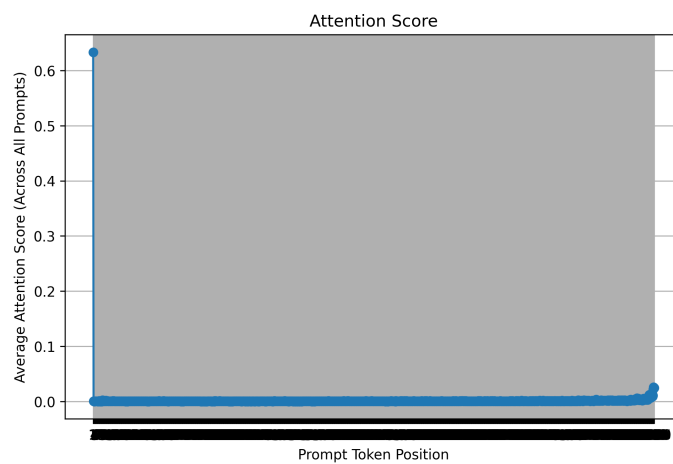
Attention Sinks Analysis (2 prompts used)



(a) Gold document at 1st position



(b) Gold document at 5th position



(c) Gold document at 10th position

Figure 4: Attention Sink analysis results, here 2 prompts has been used

3 Non-Factoid LIM Findings

3.1 Overview

This section presents findings related to non-factoid queries and responses within the LIM framework. Non-factoid items typically involve open-ended, explanatory, or opinion-based content rather than single verifiable facts. We use the nugget-based approach which was defined in Pradeep et al.[4], which used sentence nuggets to check for factual correctness of a particular output for a query. We further extend the usage of this methodology to check for factual correctness of model outputs with respect to a set of automated relevance judgements, called qrels (which were created specifically for TREC RAG 24, by the track organizers), which will exist in the main document context and will be moved across various positions.

For the non-relevant documents, we use the 2 defined approaches in Cuconasu et al. [1], where the non-relevant documents were distributed in 2 types:

1. **Discriminator:** Documents that are semantically similar to a query, but do not contain the correct answer.
2. **Noise:** Random documents, that are neither related to the query nor contain the answer.

3.2 Methodology

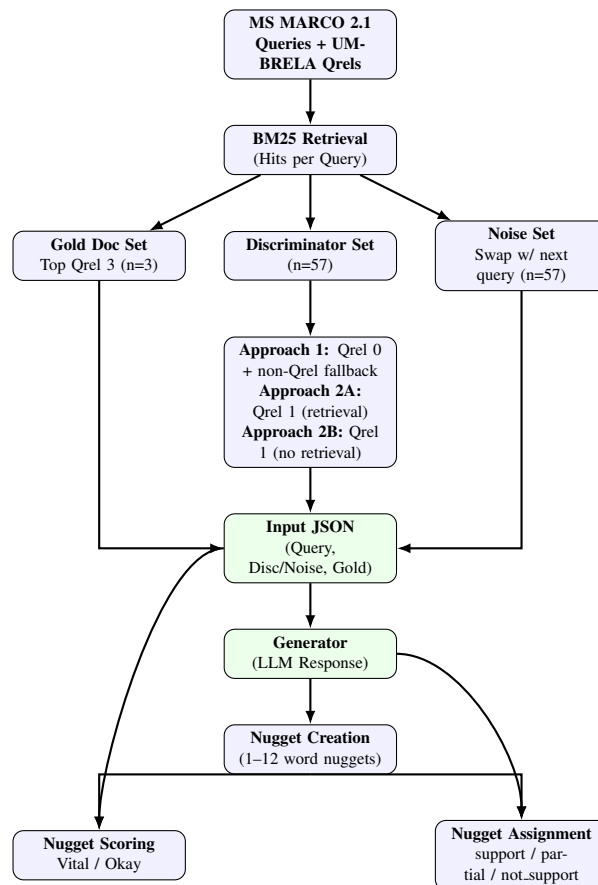


Figure 5: Overview of the dataset construction and evaluation pipeline: document set creation (Gold, Discriminator, Noise), input JSON assembly, response generation, and AutoNuggetizer-based evaluation.

Dataset Creation Pipeline

We use the MS MARCO 2.1 segmented dataset, which is provided publicly by the TREC RAG organizers. The query set and the UMBRELA qrel set for the queries is also provided by the track organizers [6]. The qrels are divided in 4 types, defined by a score label [6]. The score label basically indicates the relation between the query and a particular passage in the source dataset [6].

Score	Description
0	Represents that the passage has nothing to do with the query.
1	Represents that the passage seems related to the query but does not answer it.
2	Represents that the passage has some answer for the query, but the answer may be a bit unclear, or hidden amongst extraneous information.
3	Represents that the passage is dedicated to the query and contains the exact answer.

Table 6: UMBRELA qrel Scoring Rubric [6]

We use an external retrieving method to retrieve relevant documents for every query. For Sparse Retrieval, we will use BM25 ranking algorithm. After we get a retrieval hits set for each query, we then create a input json for the main pipeline. This json includes 3 things:

1. The query text
2. The discriminator/noise document text set
3. The gold document text set

This input json is basically a subset of the retrieval hits output, where we fix the number of discriminator documents and the number of gold documents.

Gold Document set creation

For creating the set of gold documents for a particular query, we check the retrieval hits set for a query, and search for those documents with the highest retrieval score, and also existing in the qrel 3 set for that respective query. We repeat this process until the length of the gold document set is 3, while ensuring that no duplicate documents are being inserted in the gold document set. Queries which did not have at least 3 qrel 3 documents were dropped from the final json

We chose this approach because the UMBRELA qrel rubric clearly describes the qrel 3 documents to contain the actual required answer to its respective query.

Discriminator Document set creation

For creating the set of discriminator documents, 2 approaches were thought out:

1. **Qrel 0 + Non-Qrel Discriminator Documents:** Check the retrieval hits set for a particular query, and search for those documents with the highest retrieval score, and also existing in the qrel 0 set for that respective query. Repeat this process until the length of the discriminator document set reaches

57. If the hits set exhausts and the length is still not 57, then as fallback, we take the documents with the least amount of retrieval score, which do not exist in any of qrel 0,1,2,3 for a particular query.

- 2A. **Qrel 1 Discriminator Documents:** Same as 1, except we only prioritise qrel 1, and drop those queries for which the discriminator document set, after traversing through the retrieval hits list is still not having a length of 57.
- 2B. **Qrel 1 Discriminator Documents (Without Retrieval):** An extended version of 2A, where we do not rely on any external retriever and directly take the qrel 1 documents of queries. If any query has less than 57 qrel 1 documents, we drop the query, and if any query has more than 57 qrel 1 documents, we take the top-57 listed qrel documents.

We initially started out with approach 1, and later after checking the approach for generating qrels in TREC RAG 24, defined in Upadhyay et al. [6], decided to also check how the behaviour changes when we prioritise qrel 1 documents as discriminators instead of qrel 0 documents, since Upadhyay et al. [6] mentions that the qrels with score 1 consists of documents, which even though are semantically related with their respective query, are still not actually answering it. This is exactly in line with the definition of discriminator, that has been defined by Cuconasu et al. [1]. In contrast, qrel 0 consists of documents that basically act as noise documents for their respective queries since they have no semantic relation with their query

For now in both scenarios of CASE 2, we are strictly keeping all discriminator documents to be from qrel 1, but later, based on the results we get through them, we also plan to implement the fallback logic, which was implemented in CASE 1.

Noise Document set creation

Finally, if we are working with Noise documents instead of discriminators, we generate noise documents for a particular query, by just replacing the discriminator documents of a particular query with the discriminator documents of the next query (if the query is the last mentioned query, then we take the discriminator documents of the 1st query in the list). We have already checked for similarity among succeeding queries and can confirm that each query listed is very much separate than its succeeding counterpart, implying no possible semantic similarity between a particular query and the discriminator documents of its succeeding counterpart.

AutoNuggetizer Framework Pipeline

We use the AutoNuggetizer Framework, defined in Pradeep et al. [4], for nugget creation and evaluation. The entire AutoNuggetizer framework, can be broken into 4 parts [4]:

1. **Nugget Creation:** For a given set of gold documents for any particular query, we define a set of nuggets which capture essential semantic aspects of any respective gold document. Each nugget is of 1-12 words in length.
2. **Nugget Scoring:** For any particular query, each nugget in the set of nuggets generated at the Nugget Creation stage is labelled with respect to their relevance for the query. The labels are [4]:
 - A. **Vital:** Vital nuggets represent concepts that must be present in a “good” answer.
 - B. **Okay:** Okay nuggets contribute worthwhile information about the target but are not essential.
3. **Nugget Assigner:** The Nugget set for any particular query generated in the Nugget Creation stage, is provided to the LLM as an input to perform nugget assignment with respect to a particular LLM

generated reponse of a query. Here we try to determine whether each nugget appears in the response. That is: **Does a system’s answer contain this particular nugget?**

The assignment is done by labelling the nuggets in the Nugget set with the following possible Nuggets:

- A. support: A nugget that is fully captured in the passage is labeled as support.
- B. partial_support: A nugget that is partially captured in the passage is labeled a partial_support.
- C. not_support: If the nugget is not captured at all, it is labelled as not_support

We integrate these parts of the framework in a custom pipeline of our own, where we additionally add a Generator at the starting, which basically generates the response of any LLM model for a particular query, and which is then parsed through the abovementioned stages.

Parameter	Description
Source Dataset	MS Marco 2.1 (segmented)
Dataset Size	Approach 1: 124, Approach 2A: 106, Approach 2B: 178
Individual Input Context Lengths (Document count)	60 ($\sim 17K$ tokens)
Gold Document count	3
Non-Gold Document count	57
Generator Model	Mistral 7b Instruct v0.3 (Context Length: 32K)
Nugget Creator Model	Mistral 7b Instruct v0.3
Nugget Scoring Model	Mistral 7b Instruct v0.3
Nugget Assigner Model	Mistral 7b Instruct v0.3

Table 7: Experimental setup details.

Note: The choice of the generator model was done based on the findings of Veseli et al. [7], which showcase that the Lost in the Middle effect for a particular model is the strongest when the input prompt occupies half of the context window of the model.

Evaluation Metrics

AutoNuggetizer Framework has its own set of metrics which can be used to check the factual and retrieval accuracy of a model. [4]. We have computed the following metrics for each case:

1. All Score: This score is the average of the scores for all nuggets in an answer. Given:

$$s_i = \begin{cases} 1 & \text{if assignment} = \text{support} \\ 0.5 & \text{if assignment} = \text{partial_support} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

The “All” score is then calculated as:

$$A = \frac{\sum_i s_i}{N_{\text{nuggets}}} \quad (6)$$

2. All Strict Score: This variant is calculated the same as the “All” score above, but with stricter nugget matching:

$$ss_i = \begin{cases} 1 & \text{if assignment} = \text{support} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

$$A_{\text{strict}} = \frac{\sum_i ss_i}{N_{\text{nuggets}}} \quad (8)$$

3. Vital Score: This score is the average of the scores for only the vital nuggets in the answer, where n_v represents the subset of the vital nuggets. We define s_i^v in the same exact way as s_i above, but only over the vital nuggets:

$$V = \frac{\sum_i s_i^v}{|n_v|} \quad (9)$$

4. Vital Strict: This score is used as the primary metric in the evaluation. It is calculated the same as V , but with stricter nugget matching. We define ss_i^v in the same exact way as ss_i above, but only over the vital nuggets:

$$V_{\text{strict}} = \frac{\sum_i ss_i^v}{|n_v|} \quad (10)$$

5. Weighted Score: Here we assign a weight of 1 to vital nuggets and 0.5 to okay nuggets. We define n_v to be the vital nuggets and n_o to be the okay nuggets. The score s_i^v is defined the same as above; the score s_i^o is defined similarly, but over okay nuggets. Then:

$$W = \frac{\sum s_i^v + 0.5 \cdot \sum s_i^o}{|n_v| + 0.5 \cdot |n_o|} \quad (11)$$

6. Weighted Score Strict: This variant is calculated the same way as the Weighted Score, but using the stricter nugget matching, i.e., ss_i^v and its counterpart ss_i^o .

$$W_{\text{strict}} = \frac{\sum ss_i^v + 0.5 \cdot \sum ss_i^o}{|n_v| + 0.5 \cdot |n_o|} \quad (12)$$

For final evaluation, we compute the average of these individual scores across the entire dataset.

3.3 Findings

We have implemented different variations of the setup based on the different approaches that we defined previously for generating discriminators.

We have already completed the analysis using Approach 1 and 2A and the analysis for Approach 2B is currently in progress.

Approach 1 showcases a very strong primacy but a very weak recency, which makes it difficult to state the results of Approach 1 to actually showcase Lost in the Middle effect. Possible reasoning for this could include the existence of non-discriminator documents in cases where the count of qrel 0 documents (which basically act as noise documents, so are not possibly contributing to this) is actually very less, which might contain information that is influencing the model’s output, even though they are chosen with the descending amount of relevance.

Both Approach 1 and 2A also indicate higher scores across all the metrics when we consider Discriminators as non-gold documents instead of Noise. This is consistent with the descriptions of the 2, since Discriminators, even though are not answering a particular query directly, still contain semantic similarity to the query, which leads to model to give answers with much more true surrounding context.

Approach 2A showcases considerably stronger recency when we consider discriminators as the non-gold documents instead of noise. This particular result of Approach 2A also therefore shows a clear Lost in the Middle curve indicating that the effect is clearly existent for non-factoid outputs as well, which are observable under certain conditions.

Metric	0	6	13	19	25	32	38	44	51	57
All Score	0.6498	0.4660	0.4550	0.4535	0.4537	0.4523	0.4342	0.4687	0.4726	0.4942
All Strict Score	0.5522	0.3391	0.3293	0.3314	0.3361	0.3356	0.3128	0.3553	0.3532	0.3799
Vital Score	0.7750	0.5909	0.5880	0.5775	0.5774	0.5740	0.5585	0.5944	0.5997	0.6038
Vital Strict Score	0.6856	0.4577	0.4592	0.4462	0.4536	0.4619	0.4280	0.4827	0.4708	0.4828
Weighted Score	0.6928	0.5092	0.4990	0.4953	0.4952	0.4929	0.4762	0.5108	0.5139	0.5322
Weighted Strict Score	0.5987	0.3799	0.3706	0.3718	0.3760	0.3769	0.3524	0.3980	0.3932	0.4161

Table 8: Approach 1 scores by position of the first gold document in the context (Discriminator as Non-Gold Documents), Mistral-7B-Instruct-v0.3.

Metric	0	6	13	19	25	32	38	44	51	57
All Score	0.6166	0.4332	0.4369	0.4052	0.4467	0.4031	0.4454	0.4470	0.4435	0.4621
All Strict Score	0.5180	0.3227	0.3169	0.3111	0.3338	0.2939	0.3215	0.3346	0.3283	0.3352
Vital Score	0.7337	0.5615	0.5606	0.5306	0.5671	0.5298	0.5619	0.5554	0.5626	0.5840
Vital Strict Score	0.6422	0.4445	0.4360	0.4300	0.4531	0.4099	0.4326	0.4432	0.4459	0.4534
Weighted Score	0.6540	0.4730	0.4770	0.4449	0.4849	0.4418	0.4831	0.4815	0.4795	0.5002
Weighted Strict Score	0.5577	0.3602	0.3552	0.3488	0.3703	0.3289	0.3572	0.3682	0.3633	0.3711

Table 9: Approach 1 scores by position of the first gold document in the context (Noise as Non-Gold Documents)

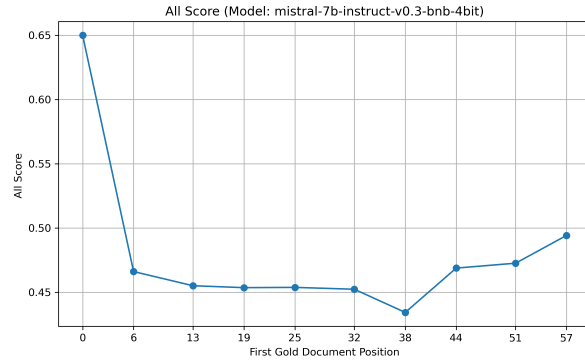
Metric	0	6	13	19	25	32	38	44	51	57
All Score	0.6120	0.4491	0.4305	0.4477	0.4309	0.4303	0.4392	0.4313	0.4669	0.4888
All Strict Score	0.5034	0.3161	0.3036	0.3222	0.3096	0.3049	0.3060	0.3044	0.3524	0.3632
Vital Score	0.7312	0.5680	0.5455	0.5636	0.5523	0.5416	0.5702	0.5669	0.5959	0.6195
Vital Strict Score	0.6455	0.4370	0.4245	0.4404	0.4318	0.4234	0.4298	0.4433	0.4826	0.4992
Weighted Score	0.6516	0.4886	0.4686	0.4863	0.4699	0.4682	0.4803	0.4750	0.5083	0.5315
Weighted Strict Score	0.5483	0.3541	0.3405	0.3598	0.3472	0.3437	0.3442	0.3455	0.3936	0.4077

Table 10: Approach 2A scores by position of the first gold document in the context (Discriminator as Non-Gold Documents), Mistral-7B-Instruct-v0.3.

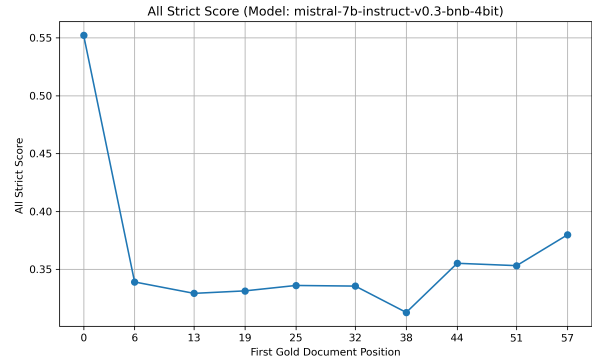
Metric	0	6	13	19	25	32	38	44	51	57
All Score	0.5626	0.3301	0.3208	0.3265	0.3113	0.3402	0.3060	0.3333	0.3399	0.3534
All Strict Score	0.4698	0.2187	0.2093	0.2168	0.2119	0.2322	0.2110	0.2363	0.2411	0.2312
Vital Score	0.6594	0.4232	0.4104	0.4192	0.4015	0.4346	0.3933	0.4405	0.4396	0.4529
Vital Strict Score	0.5772	0.3115	0.2941	0.2999	0.2900	0.3339	0.2997	0.3421	0.3387	0.3324
Weighted Score	0.5967	0.3583	0.3480	0.3560	0.3398	0.3717	0.3339	0.3627	0.3695	0.3812
Weighted Strict Score	0.5060	0.2464	0.2345	0.2430	0.2370	0.2637	0.2383	0.2651	0.2684	0.2574

Table 11: Approach 2A scores by position of the first gold document in the context (Noise as Non-Gold Documents)

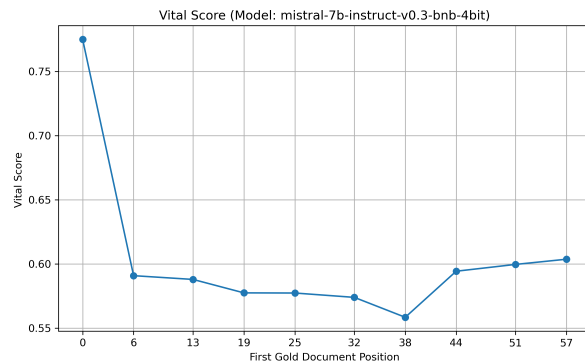
Approach 1: Discriminator as Non-Gold Documents



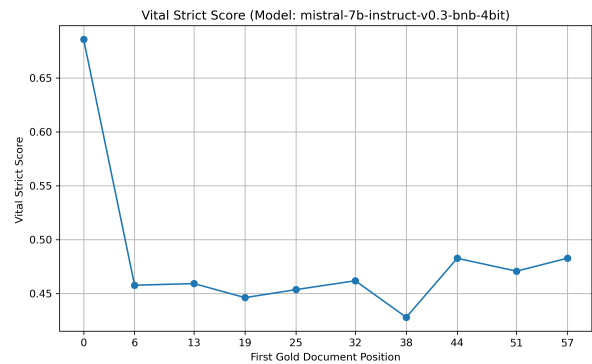
(a) All Score



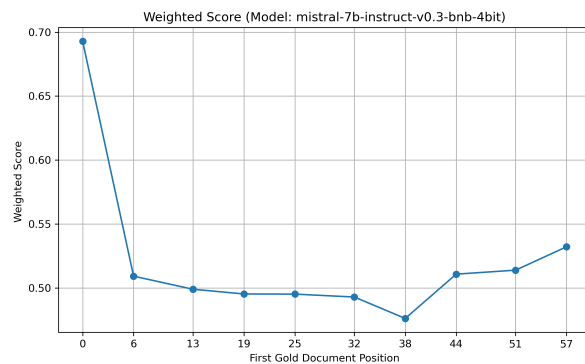
(b) All Strict Score



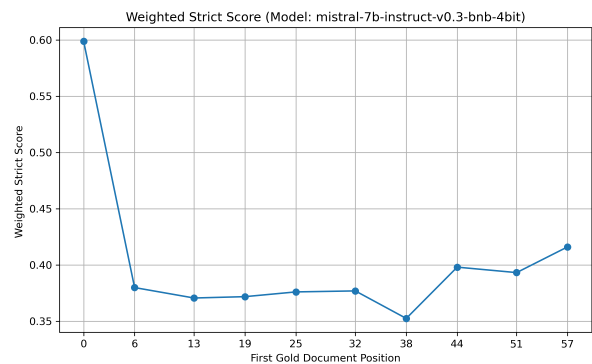
(c) Vital Score



(d) Vital Strict Score



(e) Weighted Score



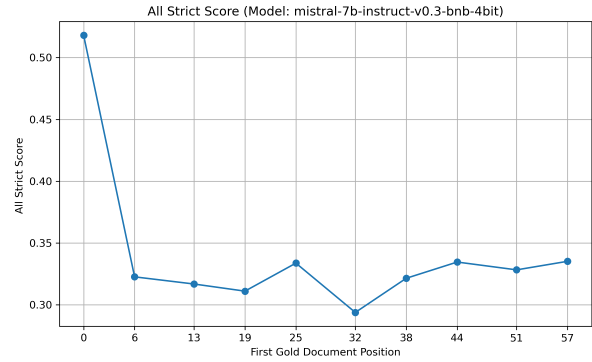
(f) Weighted Strict Score

Figure 6: Evaluation metrics for Approach 1 (Discriminator as Non-Gold Documents).

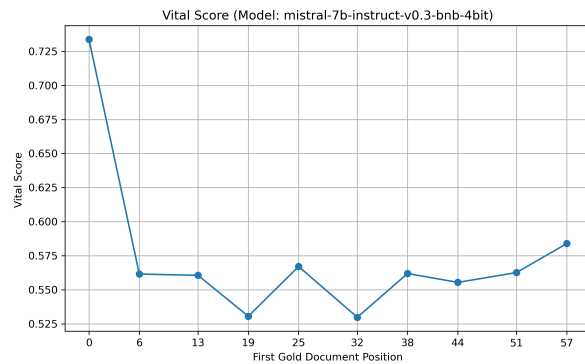
Approach 1: Noise as Non-Gold Documents



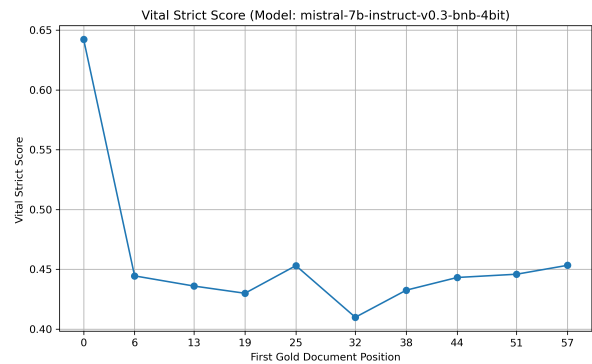
(a) All Score



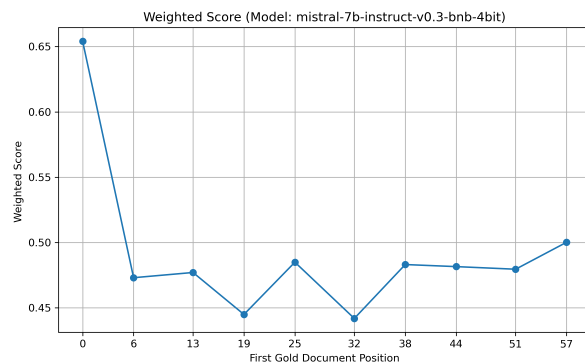
(b) All Strict Score



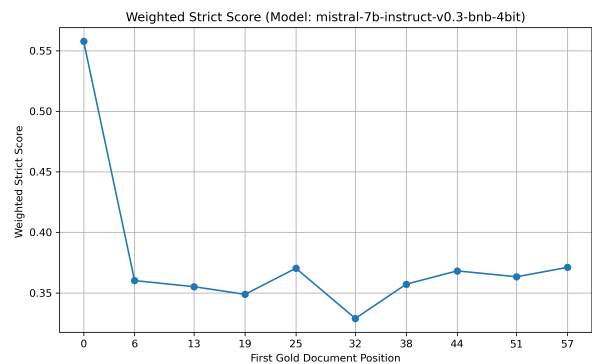
(c) Vital Score



(d) Vital Strict Score



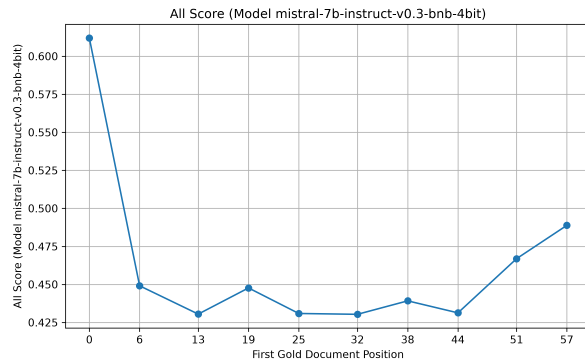
(e) Weighted Score



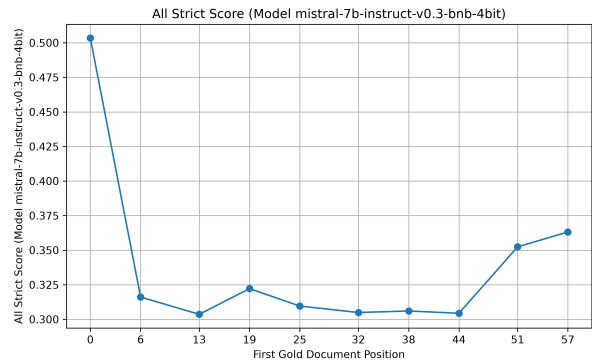
(f) Weighted Strict Score

Figure 7: Evaluation metrics for Approach 1 (Discriminator as Non-Gold Documents).

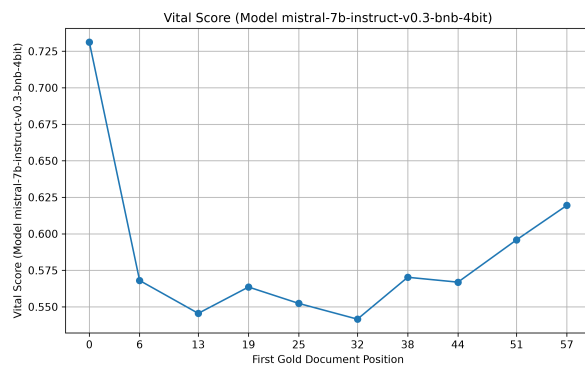
Approach 2A: Discriminator as Non-Gold Documents



(a) All Score



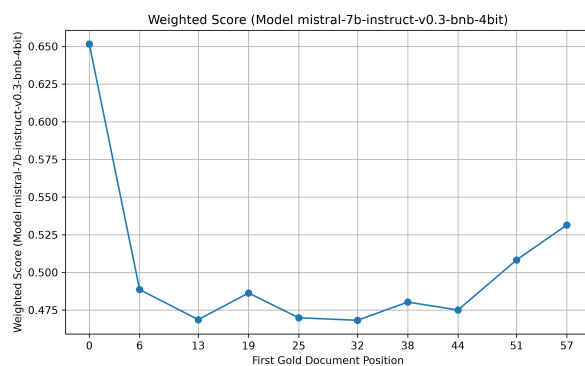
(b) All Strict Score



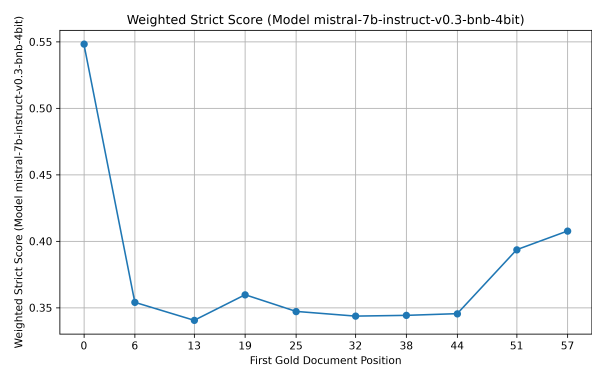
(c) Vital Score



(d) Vital Strict Score



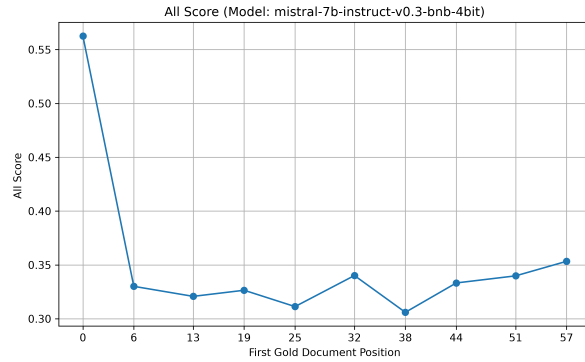
(e) Weighted Score



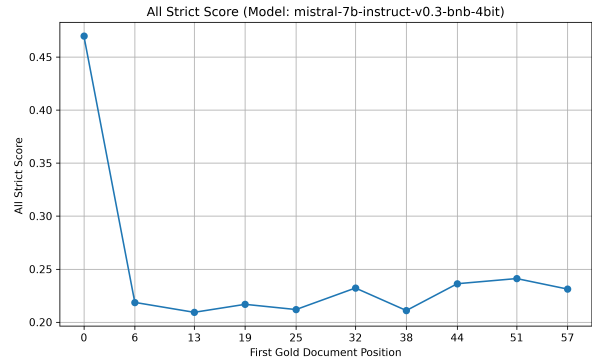
(f) Weighted Strict Score

Figure 8: Evaluation metrics for Approach 2A (Discriminator as Non-Gold Documents).

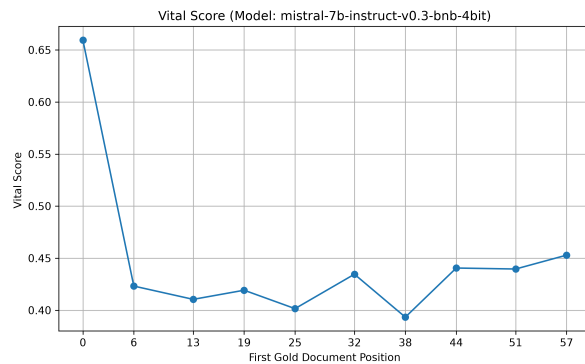
Approach 2A: Noise as Non-Gold Documents



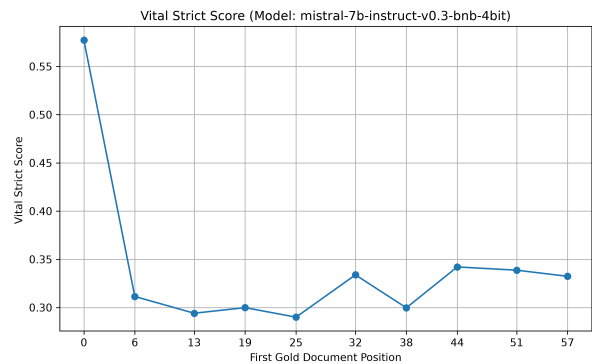
(a) All Score



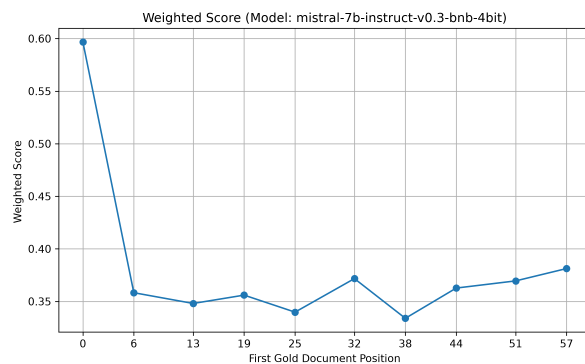
(b) All Strict Score



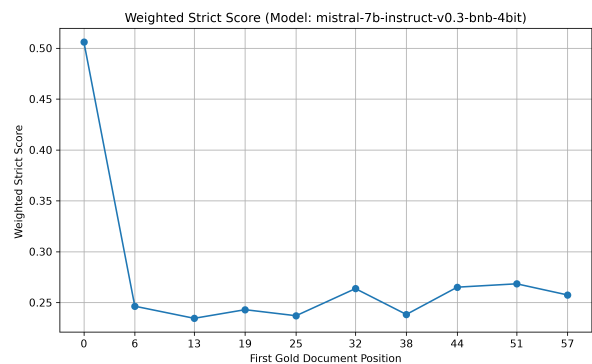
(c) Vital Score



(d) Vital Strict Score



(e) Weighted Score



(f) Weighted Strict Score

Figure 9: Evaluation metrics for Approach 2A (Noise as Non-Gold Documents).

References

- [1] Florin Cuconasu et al. “The Power of Noise: Redefining Retrieval for RAG Systems”. en. In: (2024). DOI: 10.1145/3626772.3657834. URL: <https://dl.acm.org/doi/10.1145/3626772.3657834>.
- [2] Xiangming Gu et al. “When Attention Sink Emerges in Language Models: An Empirical View”. In: arXiv:2410.10781 (Mar. 2025). arXiv:2410.10781 [cs.CL]. DOI: 10.48550/arXiv.2410.10781. URL: <http://arxiv.org/abs/2410.10781>.
- [3] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: arXiv:2307.03172 (Nov. 2023). arXiv:2307.03172. DOI: 10.48550/arXiv.2307.03172. URL: <http://arxiv.org/abs/2307.03172>.
- [4] Ronak Pradeep et al. “Initial Nugget Evaluation Results for the TREC 2024 RAG Track with the AutoNuggetizer Framework”. In: arXiv:2411.09607 (Nov. 2024). arXiv:2411.09607. DOI: 10.48550/arXiv.2411.09607. URL: <http://arxiv.org/abs/2411.09607>.
- [5] Nikolaus Salvatore, Hao Wang, and Qiong Zhang. “Lost in the Middle: An Emergent Property from Information Retrieval Demands in LLMs”. In: arXiv:2510.10276 (Oct. 2025). arXiv:2510.10276. DOI: 10.48550/arXiv.2510.10276. URL: <http://arxiv.org/abs/2510.10276>.
- [6] Shivani Upadhyay et al. “UMBRELA: Umbrela is the (Open-Source Reproduction of the) Bing RElevance Assessor”. In: arXiv:2406.06519 (2024). arXiv:2406.06519. DOI: 10.48550/arXiv.2406.06519. URL: <http://arxiv.org/abs/2406.06519>.
- [7] Blerta Veseli et al. “Positional Biases Shift as Inputs Approach Context Window Limits”. In: arXiv:2508.07479 (Aug. 2025). arXiv:2508.07479 [cs.CL]. DOI: 10.48550/arXiv.2508.07479. URL: <http://arxiv.org/abs/2508.07479>.